

Voyager

Technical Specification

Version 1.0 | June 2026 | Confidential

This document is the technical blueprint for the V1 build of Voyager — an AI-powered travel planning and trip companion web application. It defines the system architecture, database schema, API integrations, AI prompt structure, payment flow, component architecture, push notification system, and PWA configuration. This specification is the primary reference for Claude Code during the build phase.

Section	Title
1	System Architecture
2	Tech Stack & Infrastructure
3	Database Schema
4	AI Prompt Architecture
5	Booking Integration Layer
6	Payment Flow
7	Component Architecture
8	Push Notification System
9	PWA Configuration
10	Environment Variables & Configuration
11	Error Handling & Fallback Logic
12	Security Considerations

1. System Architecture

Voyager is a web-first application built on a modern serverless stack. The architecture prioritises developer speed, deployment simplicity, and the ability to iterate rapidly without infrastructure overhead. All components are cloud-managed — no servers to provision or maintain.

1.1 High-Level Architecture

Layer	Technology	Role
Frontend	Next.js 14 (App Router) + Tailwind CSS	UI rendering, routing, server components, PWA
Backend / API	Next.js API Routes + Server Actions	Business logic, AI calls, Stripe, external APIs
Database	Supabase (PostgreSQL)	All persistent data — users, trips, bookings, notifications
Authentication	Supabase Auth	Email/password and OAuth (Google). JWT session management.
AI Layer	Claude API via Vercel AI SDK	Itinerary generation, Voyager Ask, conversation mode, share sheet parsing
Payments	Stripe	Unlock fee collection. Dynamic charge calculated server-side.
Hosting	Vercel	Automatic deployments from GitHub. Edge network. Preview URLs per branch
File Storage	Supabase Storage	User-uploaded assets if needed (e.g. trip photos in future versions)
Push Notifications	Web Push API + Supabase Edge Functions	Booking window alerts, day-ahead summaries, confirmations

1.2 Data Flow — Itinerary Generation

The core product flow from trip input to unlocked itinerary follows this sequence:

Step	Action	Component
1	User completes onboarding — destination, dates, party size, travel style	Frontend form / conversation mode
2	Form data POSTed to /api/generate-itinerary server action	Next.js Server Action
3	Server action validates inputs, constructs AI prompt with trip parameters	prompt-builder.ts
4	Claude API called with system prompt + user trip data. Response streamed	ai-service.ts (abstraction layer)
5	AI returns structured itinerary JSON — days, activities, accommodation suggestions, transport legs, estimated costs	Claude API, transport APIs
6	Itinerary JSON stored in Supabase trips table with status = preview	Supabase DB
7	Trip value extracted from AI response. Complexity multiplier calculated server-side	ai-service.ts
8	Preview screen rendered — locked days + one unlocked sample day + itinerary	Frontend
9	User pays — Stripe charge processed server-side. Trip status updated to booked	Stripe + Supabase
10	Full itinerary rendered. Booking deep links constructed per element.	Frontend + booking-service.ts

1.3 AI Abstraction Layer

All Claude API calls are routed through a single abstraction layer (ai-service.ts). This decouples the product from any specific model — the model can be changed, tiered by use case, or swapped entirely without touching product code. The abstraction layer handles model selection, token limits, error handling, and retry logic.

```
// ai-service.ts – all AI calls go through here
const AI_CONFIG = {
  itinerary: { model: 'claude-sonnet-4-6', max_tokens: 8000 },
  ask: { model: 'claude-sonnet-4-6', max_tokens: 2000 },
  conversation: { model: 'claude-sonnet-4-6', max_tokens: 1000 },
  parse_booking: { model: 'claude-haiku-4-5-20251001', max_tokens: 1000 }, // cheaper for parsing
}

export async function callAI(type: keyof typeof AI_CONFIG, prompt: string, system: string) {
  const config = AI_CONFIG[type]
  // Vercel AI SDK call – streaming where appropriate
}
```

1.4 External Integration Architecture

All external booking providers are accessed via a booking abstraction layer (booking-service.ts). This layer determines which provider to surface based on booking type and destination, constructs the deep link or API call, and returns a standardised booking object to the frontend. The frontend never calls external providers directly.

```
// booking-service.ts
export function getBookingLink(type: BookingType, params: BookingParams): BookingLink {
  switch(type) {
    case 'flight': return duffelLink(params)
    case 'hotel': return duffelStaysLink(params) ?? bookingComLink(params)
    case 'rail': return railLink(params) // routes to Omio or Trip.com by destination
    case 'car': return duffelCarsLink(params)
    case 'activity': return activityLink(params) // Viator or GetYourGuide
  }
}

function railLink(params: BookingParams): BookingLink {
  const chinaIndia = ['CN', 'IN'].includes(params.destinationCountry)
  return chinaIndia ? tripComRailLink(params) : omioLink(params)
}
```

2. Tech Stack & Infrastructure

2.1 Core Dependencies

Package	Version	Purpose
next	14.x	Framework — App Router, Server Components, Server Actions, API Routes
react	18.x	UI library
tailwindcss	3.x	Utility CSS — design system tokens mapped to Tailwind config
@supabase/supabase-js	2.x	Supabase client — DB queries, auth, realtime subscriptions
@supabase/auth-helpers-nextjs	0.x	Supabase auth integration for Next.js App Router
ai	3.x	Vercel AI SDK — streaming AI responses, useChat hook
@anthropic-ai/sdk	0.x	Anthropic SDK — direct API calls where streaming not needed
stripe	14.x	Stripe Node SDK — server-side payment processing
@stripe/stripe-js	2.x	Stripe browser SDK — payment element UI
zod	3.x	Schema validation — API inputs, AI response parsing
date-fns	3.x	Date manipulation — itinerary date handling, countdowns
web-push	3.x	Web Push API — push notification sending from Edge Functions

2.2 Repository Structure

```

voyager/
■■■ app/ # Next.js App Router
■ ■■■ (auth)/ # Auth group – login, signup, forgot password
■ ■■■ (app)/ # Authenticated app group
■ ■ ■■■ home/ # Multi-trip home screen
■ ■ ■■■ trip/[tripId]/ # Active/upcoming trip view
■ ■ ■ ■■■ itinerary/ # Itinerary tab
■ ■ ■ ■■■ bookings/ # Bookings tab
■ ■ ■ ■■■ ask/ # Voyager Ask tab
■ ■ ■ ■■■ trip-settings/ # Trip tools tab
■ ■ ■■■ trip/[tripId]/past/ # Read-only past trip view
■ ■■■ onboarding/ # Trip creation – 3 paths
■ ■■■ preview/[tripId]/ # Preview + unlock screen
■ ■■■ api/ # API routes
■ ■■■ generate-itinerary/
■ ■■■ ask/
■ ■■■ stripe/
■ ■■■ parse-booking/
■ ■■■ push/
■■■ components/ # Shared UI components
■ ■■■ ui/ # Primitives – Button, Card, Input, Chip
■ ■■■ trip/ # Trip-specific – DayCard, BookingCard, etc.
■ ■■■ ask/ # Voyager Ask – ChatBubble, PromptCard, etc.
■ ■■■ onboarding/ # Onboarding components
■■■ lib/ # Server-side services
■ ■■■ ai-service.ts # AI abstraction layer
■ ■■■ booking-service.ts # Booking link abstraction
■ ■■■ pricing-service.ts # Complexity multiplier calculation
■ ■■■ prompt-builder.ts # AI prompt construction
■ ■■■ supabase/ # Supabase client instances
■■■ types/ # TypeScript types – Trip, Day, Booking, etc.
■■■ supabase/ # Supabase config
■ ■■■ migrations/ # SQL schema migrations
■ ■■■ functions/ # Edge Functions – push notifications
■■■ public/ # Static assets, PWA manifest, service worker

```

2.3 Deployment Pipeline

Event	Action
Push to main branch	Auto-deploy to production on Vercel. Supabase migrations run via CI.
Push to feature branch	Auto-deploy to preview URL on Vercel. Useful for testing before merge.
Pull request	Preview URL generated. Link posted to PR for review.
Environment variables	Set in Vercel dashboard. Never committed to repo. Supabase secrets managed separately.
Database migrations	Supabase CLI — supabase db push. Run manually before deploying breaking schema changes.

3. Database Schema

All data is stored in Supabase (PostgreSQL). Row Level Security (RLS) is enabled on all tables — users can only read and write their own data. Trip owners can read all traveller data within their trips. Invited travellers can read shared trip data and write their own individual activity layer.

3.1 Table Overview

Table	Purpose	Key Relationships
users	Voyager user accounts — extends Supabase auth.users	Referenced by all other tables via user_id
trips	Top-level trip record — one per trip per owner	Has many days, bookings, travellers
trip_travellers	Maps additional users to a shared trip	Belongs to trip, belongs to user
days	One row per day per trip — 28 rows for a 28-night trip	Belongs to trip, has many activities
activities	Individual itinerary items within a day	Belongs to day, tagged shared or individual
bookings	Confirmed bookings — flights, hotels, rail, cars, activities	Belongs to trip, optionally to a day
push_subscriptions	Web Push subscription objects per user device	Belongs to user
notifications	Log of sent notifications per user	Belongs to user, optionally to trip
ask_messages	Voyager Ask conversation history per trip per user	Belongs to trip, belongs to user
unlock_fees	Record of unlock fee payments	Belongs to trip, Stripe payment intent reference

3.2 Schema Definitions

users

```
create table users (  
  id uuid primary key references auth.users(id) on delete cascade,  
  email text not null unique,  
  full_name text,  
  avatar_url text,  
  created_at timestamptz default now(),  
  updated_at timestamptz default now()  
);
```

trips

```

create table trips (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null references users(id) on delete cascade,
  title text not null, -- e.g. 'London & Beyond'
  destination text not null, -- primary destination
  destinations text[], -- all destinations array
  country_codes text[], -- ISO codes for booking routing
  start_date date not null,
  end_date date not null,
  nights int generated always as (end_date - start_date) stored,
  party_size int not null default 1,
  budget_nzd numeric(10,2),
  travel_style text,
  status text not null default 'preview' -- preview | unlocked | active | completed
  check (status in ('preview','unlocked','active','completed')),
  itinerary_jsonb jsonb, -- full AI-generated itinerary
  trip_value_nzd numeric(10,2), -- AI-estimated trip value
  voyager_fee_nzd numeric(10,2), -- calculated unlock fee
  complexity_multiplier numeric(4,3), -- e.g. 1.200
  sample_day_number int, -- which day shown in preview
  stripe_payment_intent_id text,
  unlocked_at timestamptz,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

```

user_preferences

```

create table user_preferences (
  user_id uuid primary key references users(id) on delete cascade,
  home_airport text, -- IATA code, e.g. 'AKL'
  preferred_airlines text[],
  frequent_flyer jsonb, -- [{airline, number}]
  hotel_loyalty jsonb, -- [{chain, number}]
  seat_preference text, -- aisle | window | no_preference
  dietary text[], -- vegetarian, vegan, gluten_free, etc.
  food_dislikes text,
  accessibility_needs text,
  general_notes text, -- free text catch-all
  updated_at timestamptz default now()
);

```

learned_preferences

```

create table learned_preferences (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references users(id) on delete cascade,
  signal_type text not null, -- e.g. 'avoids_early_activities', 'prefers_boutique_hotels'
  confidence numeric(3,2) not null default 0.5, -- 0-1, increases with repeated signal
  evidence_count int not null default 1, -- how many trips contributed to this signal
  last_observed_at timestamptz default now(),
  created_at timestamptz default now(),
  unique(user_id, signal_type)
);

```

trip_travellers

```
create table trip_travellers (  
  id uuid primary key default gen_random_uuid(),  
  trip_id uuid not null references trips(id) on delete cascade,  
  user_id uuid not null references users(id) on delete cascade,  
  role text not null default 'traveller' -- traveller (owner handled by trips.owner_id)  
  check (role in ('traveller')),  
  invited_at timestamptz default now(),  
  joined_at timestamptz,  
  unique(trip_id, user_id)  
);
```

days

```
create table days (  
  id uuid primary key default gen_random_uuid(),  
  trip_id uuid not null references trips(id) on delete cascade,  
  day_number int not null, -- 1-indexed  
  date date not null,  
  title text, -- e.g. 'The Cotswolds'  
  location text, -- e.g. 'Burford, Oxfordshire'  
  country_code text, -- ISO for booking routing  
  notes text, -- day-level notes from AI or user  
  booking_status text not null default 'unbooked'  
  check (booking_status in ('unbooked','partial','booked')),  
  unique(trip_id, day_number)  
);
```

activities

```
create table activities (  
  id uuid primary key default gen_random_uuid(),  
  day_id uuid not null references days(id) on delete cascade,  
  trip_id uuid not null references trips(id) on delete cascade,  
  title text not null,  
  description text,  
  time_of_day text, -- e.g. '09:00', 'morning', 'evening'  
  type text, -- flight|hotel|rail|car|activity|dining|transfer  
  visibility text not null default 'shared'  
  check (visibility in ('shared','individual')),  
  assigned_user_id uuid references users(id), -- set if visibility = individual  
  booking_status text not null default 'unbooked'  
  check (booking_status in ('unbooked','booked','external')),  
  sort_order int not null default 0,  
  removed boolean not null default false, -- soft delete for user edits  
  created_at timestamptz default now()  
);
```

bookings

```

create table bookings (
  id uuid primary key default gen_random_uuid(),
  trip_id uuid not null references trips(id) on delete cascade,
  activity_id uuid references activities(id),
  day_id uuid references days(id),
  added_by_user_id uuid not null references users(id),
  type text not null
  check (type in ('flight','hotel','rail','car','activity','transfer','other')),
  title text not null,
  detail text,
  booking_ref text,
  provider text, -- internal only – never shown to user
  confirmation_date timestampz,
  check_in_date date,
  check_out_date date,
  departure_at timestampz,
  arrival_at timestampz,
  cost_nzd numeric(10,2),
  visibility text not null default 'shared'
  check (visibility in ('shared','individual')),
  assigned_user_id uuid references users(id),
  source text default 'manual'
  check (source in ('manual','share_sheet_import','duffel')),
  raw_import_text text, -- original share sheet text
  created_at timestampz default now()
);

```

push_subscriptions

```

create table push_subscriptions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references users(id) on delete cascade,
  endpoint text not null,
  p256dh text not null,
  auth_key text not null,
  device_label text, -- e.g. 'iPhone', 'Chrome on Mac'
  created_at timestampz default now(),
  unique(user_id, endpoint)
);

```

ask_messages

```

create table ask_messages (
  id uuid primary key default gen_random_uuid(),
  trip_id uuid not null references trips(id) on delete cascade,
  user_id uuid not null references users(id) on delete cascade,
  role text not null check (role in ('user','assistant')),
  content text not null,
  context_element_id uuid, -- activity_id if triggered from Tell Me More
  created_at timestampz default now()
);

```

unlock_fees

```

create table unlock_fees (
  id uuid primary key default gen_random_uuid(),
  trip_id uuid not null references trips(id),
  user_id uuid not null references users(id),
  amount_nzd numeric(10,2) not null,
  trip_value_nzd numeric(10,2) not null,
  complexity_multiplier numeric(4,3) not null,
  stripe_payment_intent_id text not null unique,
  stripe_charge_id text,
  status text not null default 'pending'
  check (status in ('pending','succeeded','refunded')),
  created_at timestamptz default now()
);

```

3.3 Row Level Security Policies

Table	Policy	Rule
trips	Select own trips	owner_id = auth.uid() OR id IN (SELECT trip_id FROM trip_travellers WHERE user_id = .
trips	Insert own trips	owner_id = auth.uid()
trips	Update own trips	owner_id = auth.uid()
days	Select trip member	trip_id in (trips user can see)
activities	Select shared or own	visibility = shared OR assigned_user_id = auth.uid()
activities	Insert own individual	assigned_user_id = auth.uid() OR trip owner
bookings	Select shared or own	visibility = shared OR assigned_user_id = auth.uid()
bookings	Insert own bookings	added_by_user_id = auth.uid()
ask_messages	Select own messages	user_id = auth.uid()
push_subscriptions	All own rows	user_id = auth.uid()

4. AI Prompt Architecture

Voyager uses Claude for four distinct AI tasks, each with its own system prompt and context injection strategy. All prompts are constructed server-side and never exposed to the client. The AI abstraction layer (ai-service.ts) handles model selection and token budgets per task type.

4.1 Itinerary Generation Prompt

The itinerary generation prompt is the most complex and highest-stakes prompt in the product. It must produce a fully structured, day-by-day itinerary in a predictable JSON format that the application can parse and store. The prompt is constructed dynamically from the user's trip parameters.

System Prompt — Itinerary Generation

You are Voyager – an expert travel planner with deep local knowledge of every destination worldwide.

You build complete, day-by-day travel itineraries that feel personally crafted and locally informed.

Your itineraries:

- Are specific, not generic. Real restaurant names, real hotel options, real local insight.
- Avoid tourist traps unless they genuinely merit inclusion. Explain why.
- Consider logistics – travel times, opening hours, booking requirements, seasonal factors.
- Include a mix of accommodation, transport, activities, and dining across each day.
- Account for the party size, travel style, and budget provided.
- Never use phrases like 'AI-generated', 'AI suggests', or similar. You are Voyager.

Output format: respond ONLY with valid JSON matching the ItinerarySchema exactly.
Do not include any text before or after the JSON block.

User Prompt — Itinerary Generation

Build a complete {nights}-night itinerary for {destination}.

Dates: {start_date} to {end_date}.

Party: {party_size} adults.

Budget: {budget} NZD total estimated spend.

Travel style: {travel_style}.

Additional preferences: {preferences}.

Return the itinerary as JSON matching this schema exactly:

```
{
  title: string, // e.g. 'London & Beyond'
  summary: string, // 2-3 sentence trip overview
  estimated_value_nzd: number, // total estimated trip cost in NZD
  sample_day_recommendation: number, // which day number best showcases the trip
  days: [
    {
      day_number: number,
      date: string, // YYYY-MM-DD
      title: string, // e.g. 'The Cotswolds'
      location: string, // specific location
      country_code: string, // ISO 3166-1 alpha-2
      activities: [
        {
          time_of_day: string, // e.g. '09:00' or 'morning'
          type: string, // flight|hotel|rail|car|activity|dining|transfer
          title: string,
          description: string, // specific, locally informed detail
          booking_required: boolean,
          booking_notes: string, // e.g. 'book 2 weeks ahead'
          estimated_cost_nzd: number
        }
      ]
    }
  ]
}
```

Preference Context Injection

Before the itinerary generation prompt is sent, explicit preferences and high-confidence learned preferences are fetched and folded into the user prompt automatically. The user never has to repeat this information per trip.

```
// prompt-builder.ts – buildPreferenceContext()
async function buildPreferenceContext(userId: string): Promise<string> {
  const explicit = await getUserPreferences(userId)
  const learned = await getLearnedPreferences(userId, { minConfidence: 0.7 })

  return `
Known preferences for this traveller:
${explicit.home_airport ? `Home airport: ${explicit.home_airport}` : ''}
${explicit.dietary?.length ? `Dietary: ${explicit.dietary.join(', ')} ` : ''}
${explicit.preferred_airlines?.length ? `Preferred airlines:
${explicit.preferred_airlines.join(', ')} ` : ''}
${explicit.general_notes ? `Notes: ${explicit.general_notes}` : ''}
${learned.map(l => `Learned: ${l.signal_type} (observed across ${l.evidence_count}
trips)`).join(' ')}
`
}

// This context is appended to the itinerary generation user prompt automatically
// Empty fields are omitted – no preference data does not degrade output quality
```

Response Parsing & Validation

The AI response is parsed using Zod schema validation. If validation fails, the AI is called again with an error correction prompt (maximum 2 retries). If retries fail, the user is shown an error and the trip is not stored. The `sample_day_recommendation` field tells Voyager which day to show unlocked in the preview — the AI selects the most compelling day, not Day 1.

4.2 Voyager Ask Prompt

Voyager Ask operates in two modes: pre-unlock (limited, general) and post-unlock (full, trip-aware). The system prompt is constructed differently for each mode.

System Prompt — Voyager Ask (Post-unlock)

You are Voyager – a knowledgeable travel companion for this specific trip.
You have full context of the user's itinerary, including their destinations, dates, accommodation, booked activities, and travel companions.

Your role:

- Answer questions about this specific trip with precision and local knowledge.
- Be proactive – if you notice something useful the user hasn't asked about, mention it.
- Keep answers concise on mobile. Lead with the most useful information.
- Never say 'as an AI' or 'I am an AI assistant'. You are Voyager.
- If asked about something outside the trip, answer helpfully but briefly.

Trip context injected below:

[TRIP_CONTEXT]

Trip Context Injection

```
// prompt-builder.ts — buildAskContext()
function buildAskContext(trip: Trip, currentDay?: number): string {
  return `
Trip: ${trip.title}
Dates: ${trip.start_date} to ${trip.end_date} (${trip.nights} nights)
Party: ${trip.party_size} adults
Current day: ${currentDay ?? 'pre-trip'}
Destinations: ${trip.destinations.join(', ')}

Itinerary summary:
${trip.days.map(d => `Day ${d.day_number}: ${d.title} — ${d.location}`).join('\n')}

Confirmed bookings:
${trip.bookings.map(b => `${b.type}: ${b.title} (${b.booking_ref ?? 'no ref'})`).join('\n')}
`
}
```

Pre-unlock Ask — System Prompt

You are Voyager — a travel planning expert.
 The user is building a trip to {destination} and has {questions_remaining} free questions.
 Answer their question with genuine local knowledge and insight.
 Keep responses concise — 3-5 sentences maximum.
 Do not mention the question limit. Do not suggest they upgrade or pay.
 Never say 'as an AI'. You are Voyager.

4.3 Conversation Mode Prompt (Onboarding)

Conversation mode in onboarding collects trip parameters through a natural dialogue. The goal is to extract the same fields as the standard form — destination, dates, party size, budget, travel style — through conversation rather than form inputs. The conversation ends when all required fields are collected.

You are Voyager — helping a traveller plan their next trip.
 Your goal is to collect the following information through natural conversation:
 Required: destination, travel dates, party size
 Optional: budget (NZD), travel style, specific interests or preferences

Guidelines:

- Ask one question at a time. Keep the conversation natural and warm.
- When you have all required fields, respond with a JSON block:

```
{collected: true, destination, start_date, end_date, party_size, budget_nzd, travel_style, preferences}
```
- Until then, respond normally to continue the conversation.
- Do not mention that you are collecting data for a form.

4.4 Share Sheet Booking Parser

When a user imports a booking confirmation via the share sheet, the raw text is sent to Claude for extraction. The parser uses the cheaper Haiku model since the task is extraction not generation. Claude parses the confirmation email text and returns a structured booking object.

System: Extract booking details from the following travel confirmation text.

Return ONLY valid JSON with these fields:

```
{
  type: 'flight'|'hotel'|'rail'|'car'|'activity'|'transfer'|'other',
  title: string,
  booking_ref: string | null,
  provider: string | null,
  detail: string,
  check_in_date: string | null, // YYYY-MM-DD
  check_out_date: string | null, // YYYY-MM-DD
  departure_at: string | null, // ISO 8601
  arrival_at: string | null, // ISO 8601
  confidence: number // 0-1 extraction confidence
}
```

If the text is not a travel booking, return {type: 'unknown', confidence: 0}.

5. Booking Integration Layer

All booking integrations are accessed via the booking-service.ts abstraction layer. The frontend never calls external providers directly. The provider powering each booking element is never surfaced to the user — the label is always the transport type (flights, train, hotel) not the provider name.

5.1 Provider Summary

Category	V1 Provider	Integration Type	Self-serve Signup
Flights	Duffel API	REST API — direct integration	Yes — duffel.com
Hotels (primary)	Duffel Stays API	REST API — direct integration	Yes — duffel.com
Hotels (fallback)	Booking.com affiliate	Deep links — tracked affiliate	Yes — booking.com/affiliate
Car Rental	Duffel Cars API	REST API — direct integration	Yes — duffel.com
Rail — Europe/UK/Japan/Americas/Africa	Omio affiliate	Deep links — tracked affiliate	Yes — omio.com/partners
Rail — China/India	Trip.com affiliate	Deep links — tracked affiliate	Yes — trip.com/partners
Activities	Viator + GetYourGuide	Deep links — tracked affiliate	Yes — both have affiliate programmes
Airbnb	External only	Share sheet import	N/A — Airbnb API closed

5.2 Deep Link Construction

Deep links are constructed server-side with all relevant parameters pre-filled. The user taps a booking element and lands on a pre-populated search or listing page — no re-entry of destination, dates, or party size.

Omio Rail Deep Link

```
function omioLink(params: RailParams): string {
  const base = 'https://www.omio.com/results'
  const q = new URLSearchParams({
    origin: params.origin,
    destination: params.destination,
    outboundDate: params.date, // YYYY-MM-DD
    adults: String(params.adults),
    ref: OMIO_AFFILIATE_ID,
  })
  return `${base}?${q.toString()}`
}
```

Trip.com Rail Deep Link (China/India)

```
function tripComRailLink(params: RailParams): string {
  const base = 'https://www.trip.com/trains/list'
  const q = new URLSearchParams({
    fromStation: params.origin,
    toStation: params.destination,
    departDate: params.date,
    adults: String(params.adults),
    locale: 'en-XX',
    allianceid: TRIPCOM_AFFILIATE_ID,
  })
  return `${base}?${q.toString()}`
}
```

Booking.com Hotel Deep Link

```
function bookingComLink(params: HotelParams): string {
  const base = 'https://www.booking.com/searchresults.html'
  const q = new URLSearchParams({
    ss: params.destination,
    checkin: params.checkIn, // YYYY-MM-DD
    checkout: params.checkOut,
    group_adults: String(params.adults),
    no_rooms: String(params.rooms ?? 1),
    aid: BOOKINGCOM_AFFILIATE_ID,
  })
  return `${base}?${q.toString()}`
}
```

5.3 Duffel API Integration

Duffel handles flights, hotels, and car rental via direct API integration. Unlike the affiliate deep link providers, Duffel allows Voyager to search and initiate bookings in-app. The user completes payment through Duffel's hosted checkout or Voyager's payment layer.

```
// lib/duffel.ts
import { Duffel } from '@duffel/api'
const duffel = new Duffel({ token: process.env.DUFFEL_API_KEY })

// Flight search – called when user taps 'Book flight' on an itinerary element
export async function searchFlights(params: FlightSearchParams) {
  const response = await duffel.offerRequests.create({
    slices: params.slices, // origin/destination/date per leg
    passengers: params.passengers,
    cabin_class: params.cabinClass ?? 'economy',
  })
  return response.data
}
```

5.4 Share Sheet Import Flow

When a user shares a booking confirmation to Voyager from their email or any other app, the following flow runs:

Step	Action
1	User opens confirmation email and uses iOS/Android share sheet to share to Voyager
2	Voyager receives the shared text via Web Share Target API (registered in PWA manifest)
3	Raw text POSTed to /api/parse-booking
4	Server sends text to Claude Haiku with the booking parser prompt
5	Claude returns structured booking JSON with confidence score
6	If confidence > 0.7: booking saved to bookings table, day updated, user notified
7	If confidence < 0.7: user shown parsed result with edit fields before saving
8	Push notification sent: 'Booking confirmed — [title] added to your trip'

6. Payment Flow

6.1 Fee Calculation

The Voyager fee is calculated entirely server-side from the AI-generated itinerary. The client never calculates or influences the fee. The calculation runs in pricing-service.ts.

```
// pricing-service.ts
const BASE_RATE = 0.0075 // 0.75%
const MAX_MULTIPLIER = 1.5

function calculateMultiplier(trip: TripParams): number {
  const duration = durationScore(trip.nights) // 1.0 - 1.3
  const dests = destinationScore(trip.destinations.length, trip.countryCount) // 1.0 - 1.3
  const partySize = partySizeScore(trip.partySize) // 1.0 - 1.3
  const complexity = complexityScore(trip.transportModes, trip.accommodationTypes) // 1.0 - 1.3
  const avg = (duration + dests + partySize + complexity) / 4
  return Math.min(avg, MAX_MULTIPLIER)
}

export function calculateFee(tripValueNZD: number, trip: TripParams): FeeResult {
  const multiplier = calculateMultiplier(trip)
  const fee = Math.round(tripValueNZD * BASE_RATE * multiplier * 100) / 100
  return { fee, multiplier, tripValueNZD }
}

function durationScore(nights: number): number {
  if (nights < 7) return 1.0
  if (nights < 15) return 1.1
  if (nights < 29) return 1.2
  return 1.3
}
```

6.2 Stripe Integration

Stripe is used for all payment collection. The payment intent is created server-side and the client uses Stripe's Payment Element for card entry. Voyager never handles raw card data.

Step	Code location	Action
Create payment intent	/api/stripe/create-intent	POST with trip_id. Server calculates fee, creates Stripe PaymentIntent, returns client_secret.
Render payment UI	components/unlock/PaymentForm.tsx	Stripe Payment Element rendered with client_secret. User enters card details.
Confirm payment	Stripe hosted	Stripe handles 3DS, fraud checks, card processing.
Webhook received	/api/stripe/webhook	payment_intent.succeeded event received. Trip status updated to unlocked. unlockScreen rendered.
Unlock rendered	app/trip/[tripId]/itinerary	Full itinerary shown. Booking links active. Add-on screen shown first.

6.3 Refund Policy Implementation

Refunds are handled via Stripe's API. The refund policy (documented in legal) is: full refund for technical failures before itinerary delivery; no refund once itinerary is delivered and viewed. Manual refund processing via Stripe dashboard for edge cases.

7. Component Architecture

Components follow Next.js App Router conventions — Server Components by default, Client Components (use client) only where interactivity requires it. The design system tokens from the Voyager Design System are mapped to Tailwind config.

7.1 Tailwind Design System Mapping

```
// tailwind.config.ts
theme: {
  extend: {
    colors: {
      accent: '#7F543D',
      'accent-light': '#F4EDE7',
      secondary: '#74A8A4',
      hero: '#2C4858',
      bg: '#E9E4DC',
      surface: '#FDFAF6',
      border: '#D4CCBE',
      'text-mid': '#504840',
      'text-light': '#948A80',
    },
    borderRadius: {
      sm: '12px',
      md: '20px',
      lg: '24px',
      xl: '32px',
    },
    fontFamily: {
      heading: ['Libre Baskerville', 'Georgia', 'serif'],
      body: ['Nunito', 'system-ui', 'sans-serif'],
      mono: ['ui-monospace', 'SF Mono', 'monospace'],
    },
  },
}
```

7.2 Core Component Map

Component	Type	Purpose
TripHeroCard	Client	Dark gradient trip card with booking progress bar — home screen and trip header
DayCard	Client	Collapsible day row with status indicator, activities list, edit/view actions
BookingCard	Client	Single booking item — confirmed (green) or to-book (amber). Shows ref, who tag.
BookingsDayStrip	Client	Scrollable day selector with amber dots on days with outstanding bookings
AskInterface	Client	Full Voyager Ask UI — contextual prompts feed + chat input + voice mic
ChatBubble	Client	Single message in Voyager Ask — user (right, accent) or Voyager (left, tag)
PromptCard	Client	Tappable contextual prompt suggestion in Ask feed
UnlockScreen	Client	Preview unlock — fee display, no hidden fees banner, Stripe Payment Element
AddOnCard	Client	Add-on offer card — icon, title, contextual pricing, add/added toggle
PackingList	Client	Interactive packing checklist — tap to tick, add item, share

CurrencyConverter	Client	Simple NZD → destination currency converter, exchange rate from API
WeatherWidget	Server	5-day weather for current destination — server fetched, cached
NotifBell	Client	Header notification bell with badge count, expandable alert panel
VoiceMic	Client	SVG mic button — Web Speech API for voice input in Ask and conversation mode
TripInviteLink	Server	Generates shareable trip invitation link
PastTripCard	Server	Read-only past trip archive card with highlights

8. Push Notification System

Push notifications are implemented using the Web Push API and sent from Supabase Edge Functions. The permission prompt is shown to the user once — after their first trip is unlocked. All notification logic runs server-side; the client only manages subscription registration.

8.1 Notification Types

Type	Trigger	Example Content	
Booking window alert	Scheduled check — daily at 8am. Voyager checks upcoming Bath Spa for bookings opens today. Day 8 booked (6 days to go).		
Day-ahead summary	9pm local time the evening before each travel day.	'Tomorrow: Bath. Roman Baths open at 9am — arrive early to beat the queues.'	
Booking confirmation	Immediately after successful share sheet import.	'GWR London → Bath added to Day 8.'	
Departure reminder	7 days before and 1 day before departure date.	'You leave for London in 7 days. 4 bookings still outstanding.'	
Alert — urgent	When a time-sensitive booking window opens with fewer than 3 days to go.	'Book now: Mayfield tickets released today. Limited availability.'	

8.2 Implementation

```
// supabase/functions/send-push/index.ts — Edge Function
import webpush from 'npm:web-push'

webpush.setVapidDetails(
  'mailto:' + Deno.env.get('VAPID_EMAIL'),
  Deno.env.get('VAPID_PUBLIC_KEY'),
  Deno.env.get('VAPID_PRIVATE_KEY')
)

export async function sendPush(userId: string, payload: NotificationPayload) {
  const subs = await supabase
    .from('push_subscriptions')
    .select('*')
    .eq('user_id', userId)

  for (const sub of subs.data) {
    await webpush.sendNotification(
      { endpoint: sub.endpoint, keys: { p256dh: sub.p256dh, auth: sub.auth_key } },
      JSON.stringify(payload)
    )
  }
}
```

8.3 Client-Side Subscription Registration

```
// app/hooks/usePushNotifications.ts
export async function registerPushSubscription() {
  const reg = await navigator.serviceWorker.ready
  const sub = await reg.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey: process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY
  })
  // Save subscription to Supabase push_subscriptions table
  await supabase.from('push_subscriptions').upsert({
    user_id: userId,
    endpoint: sub.endpoint,
    p256dh: arrayBufferToBase64(sub.getKey('p256dh')),
    auth_key: arrayBufferToBase64(sub.getKey('auth')),
  })
}
```

9. PWA Configuration

Voyager is a Progressive Web App — the web app can be added to the home screen on iOS and Android where it runs full-screen without browser chrome, indistinguishable from a native app. Offline capability is limited to core itinerary data for the current active trip.

9.1 Web App Manifest

```
// public/manifest.json
{
  "name": "Voyager",
  "short_name": "Voyager",
  "description": "Your personal travel expert",
  "start_url": "/home",
  "display": "standalone",
  "background_color": "#E9E4DC",
  "theme_color": "#2C4858",
  "orientation": "portrait",
  "share_target": {
    "action": "/api/parse-booking",
    "method": "POST",
    "enctype": "multipart/form-data",
    "params": { "text": "text", "title": "title" }
  },
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" },
    { "src": "/icon-maskable.png", "sizes": "512x512", "type": "image/png", "purpose":
      "maskable" }
  ]
}
```

9.2 Service Worker — Offline Strategy

```
// public/sw.js
// Cache strategy: network-first for API calls, cache-first for static assets

const CACHE_NAME = 'voyager-v1'
const OFFLINE_TRIP_CACHE = 'voyager-trip-v1'

// Cached offline for active trip:
// - /trip/[tripId]/itinerary – itinerary data
// - /trip/[tripId]/bookings – confirmed bookings
// - Hotel addresses, booking references, emergency info

self.addEventListener('fetch', event => {
  if (event.request.url.includes('/api/')) {
    // Network first – fall back to cached response if offline
    event.respondWith(networkFirst(event.request))
  } else {
    // Cache first for static assets
    event.respondWith(cacheFirst(event.request))
  }
})
```

10. Environment Variables & Configuration

Variable	Where set	Purpose
ANTHROPIC_API_KEY	Vercel (server)	Claude API authentication
NEXT_PUBLIC_SUPABASE_URL	Vercel (public)	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Vercel (public)	Supabase anonymous key for client
SUPABASE_SERVICE_ROLE_KEY	Vercel (server)	Supabase service role — server-side operations only
STRIPE_SECRET_KEY	Vercel (server)	Stripe server-side API key
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Vercel (public)	Stripe publishable key for Payment Element
STRIPE_WEBHOOK_SECRET	Vercel (server)	Stripe webhook signature verification
DUFFEL_API_KEY	Vercel (server)	Duffel API for flights, hotels, car rental
OMIO_AFFILIATE_ID	Vercel (server)	Omio affiliate tracking ID
TRIPCOM_AFFILIATE_ID	Vercel (server)	Trip.com affiliate tracking ID
BOOKINGCOM_AFFILIATE_ID	Vercel (server)	Booking.com affiliate tracking ID
VIATOR_AFFILIATE_ID	Vercel (server)	Viator affiliate tracking ID
GETYOURGUIDE_AFFILIATE_ID	Vercel (server)	GetYourGuide affiliate tracking ID
VAPID_PUBLIC_KEY	Vercel (server)	Web Push VAPID public key
VAPID_PRIVATE_KEY	Vercel (server)	Web Push VAPID private key
VAPID_EMAIL	Vercel (server)	Web Push VAPID contact email
NEXT_PUBLIC_VAPID_PUBLIC_KEY	Vercel (public)	VAPID public key for client-side subscription
WEATHER_API_KEY	Vercel (server)	Weather API for companion tools tab
EXCHANGE_RATE_API_KEY	Vercel (server)	Currency conversion for currency tool

11. Error Handling & Fallback Logic

Scenario	Handling
AI itinerary generation fails (timeout, invalid response)	Retry to 2 times with correction prompt. If still failing, show user-friendly error and offer to try again.
AI response fails Zod validation	Retry with explicit schema correction prompt. Log validation errors for monitoring.
Stripe payment fails	Stripe handles retries and card errors via Payment Element UI. Voyager shows error from Stripe. Trip is not created.
Stripe webhook not received	Voyager polls trip status for 30 seconds after payment confirmation. If webhook not received, manual check.
Duffel API unavailable	Fall back to Booking.com deep link for hotels. Log outage. Alert if sustained.
Omio deep link invalid destination	Fall back to omio.com homepage with destination pre-filled in search. Log for monitoring.
Push notification delivery fails	Log failure. Do not retry more than once. Not critical — in-app alerts always available.
Share sheet import — parsing confidence low	Show user the parsed result with edit fields. User confirms before saving. Never silently discard.
Supabase connection unavailable	Next.js error boundary shows service unavailable message. Cached trip data shown where available.
AI Ask — rate limit hit	Show 'Voyager is busy — try again in a moment' message. Do not count against question limit.

12. Security Considerations

Area	Implementation
Authentication	Supabase Auth — JWT tokens. All API routes verify auth token server-side before processing. No unauthenticated access.
Row Level Security	RLS enabled on all Supabase tables. Users can only access their own data and shared trip data they are explicitly granted.
API keys	All API keys server-side only. No API key ever exposed to the client. NEXT_PUBLIC_ variables contain only non-sensitive data.
Payment data	Voyager never handles raw card data. All payment processing through Stripe's PCI-compliant infrastructure.
AI prompt injection	User inputs sanitised before insertion into AI prompts. System prompt clearly separates user content. No user input can alter system instructions.
Webhook verification	All Stripe webhooks verified using STRIPE_WEBHOOK_SECRET before processing. Unverified webhooks rejected.
CORS	Next.js API routes restricted to same-origin. External webhooks (Stripe) verified by signature, not origin.
Share sheet input	All share sheet text treated as untrusted input. Parsed by Claude with clear instruction not to execute any instructions.
Rate limiting	AI Ask endpoints rate-limited by user ID. Prevents abuse of Claude API quota. Pre-unlock limit enforced server-side.
Environment	All secrets in Vercel environment variables. .env.local for local development only, in .gitignore. Never committed to code.

Voyager Technical Specification v1.0 — June 2026. This document should be read alongside the Voyager PRD v2.6 and Business Plan v1.5. All technical decisions documented here are the starting point for the V1 build. Decisions may evolve during development — significant changes should be reflected back into this document.